

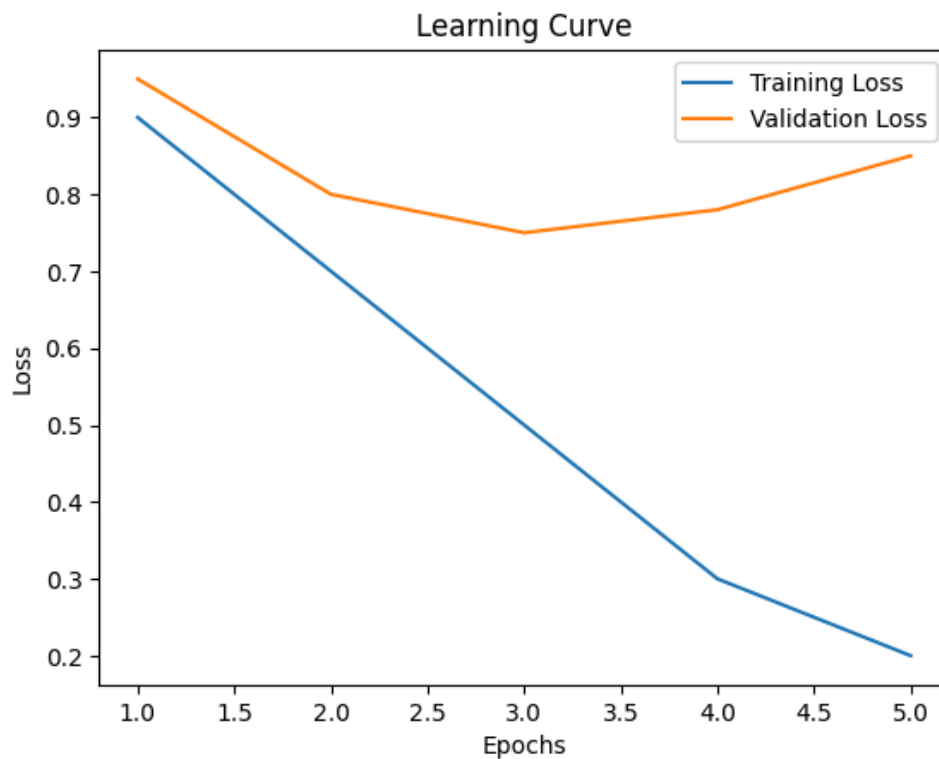
✓ Libraries

```
import tensorflow as tf
import numpy as np
import matplotlib.pyplot as plt
```

✓ Learning Curves

```
epochs = [1,2,3,4,5]
train_loss = [0.9, 0.7, 0.5, 0.3, 0.2]
val_loss = [0.95, 0.8, 0.75, 0.78, 0.85]
```

```
plt.plot(epochs, train_loss, label="Training Loss")
plt.plot(epochs, val_loss, label="Validation Loss")
plt.xlabel("Epochs")
plt.ylabel("Loss")
plt.title("Learning Curve")
plt.legend()
plt.show()
```



using tensorflow

```
#Sample Dataset
x = np.random.rand(1000, 10)
y = (np.sum(x, axis=1) > 5).astype(int)
```

```
#model building
```

```

model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)), #hidden layer
    tf.keras.layers.Dense(32, activation='relu'), #hidden layer
    tf.keras.layers.Dense(1, activation='sigmoid') #output layer
])

```

```

model.compile(
    optimizer='adam',
    loss='binary_crossentropy', # loss for binary classification
    metrics=['accuracy'] # track accuracy
)

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)

```

```

history = model.fit(
    x, y,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
)

```

```

Epoch 1/20
25/25 ————— 5s 19ms/step - accuracy: 0.5150 - loss: 0.7140 - val_accu
Epoch 2/20
25/25 ————— 0s 9ms/step - accuracy: 0.6350 - loss: 0.6752 - val_accu
Epoch 3/20
25/25 ————— 0s 8ms/step - accuracy: 0.6463 - loss: 0.6549 - val_accu
Epoch 4/20
25/25 ————— 0s 10ms/step - accuracy: 0.7550 - loss: 0.6296 - val_accu
Epoch 5/20
25/25 ————— 0s 4ms/step - accuracy: 0.7925 - loss: 0.5959 - val_accu
Epoch 6/20
25/25 ————— 0s 4ms/step - accuracy: 0.8213 - loss: 0.5561 - val_accu
Epoch 7/20
25/25 ————— 0s 4ms/step - accuracy: 0.8350 - loss: 0.5110 - val_accu
Epoch 8/20
25/25 ————— 0s 4ms/step - accuracy: 0.8675 - loss: 0.4632 - val_accu
Epoch 9/20
25/25 ————— 0s 4ms/step - accuracy: 0.8775 - loss: 0.4176 - val_accu
Epoch 10/20
25/25 ————— 0s 4ms/step - accuracy: 0.9050 - loss: 0.3705 - val_accu
Epoch 11/20
25/25 ————— 0s 4ms/step - accuracy: 0.8863 - loss: 0.3398 - val_accu
Epoch 12/20
25/25 ————— 0s 4ms/step - accuracy: 0.9262 - loss: 0.3010 - val_accu
Epoch 13/20
25/25 ————— 0s 4ms/step - accuracy: 0.9425 - loss: 0.2664 - val_accu
Epoch 14/20
25/25 ————— 0s 4ms/step - accuracy: 0.9388 - loss: 0.2429 - val_accu
Epoch 15/20
25/25 ————— 0s 4ms/step - accuracy: 0.9463 - loss: 0.2207 - val_accu
Epoch 16/20
25/25 ————— 0s 5ms/step - accuracy: 0.9538 - loss: 0.2036 - val_accu
Epoch 17/20
25/25 ————— 0s 4ms/step - accuracy: 0.9438 - loss: 0.1928 - val_accu
Epoch 18/20
25/25 ————— 0s 4ms/step - accuracy: 0.9538 - loss: 0.1768 - val_accu
Epoch 19/20
25/25 ————— 0s 4ms/step - accuracy: 0.9638 - loss: 0.1660 - val_accu
Epoch 20/20
25/25 ————— 0s 4ms/step - accuracy: 0.9613 - loss: 0.1547 - val_accu

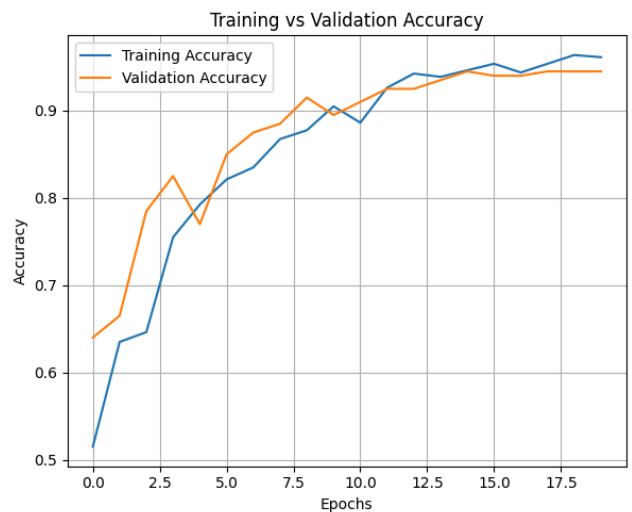
```

```
#Learning curves
train_loss = history.history['loss'] # training loss
val_loss = history.history['val_loss'] # validation loss
train_acc = history.history['accuracy'] # training accuracy
val_acc = history.history['val_accuracy'] # validation accuracy
```

```
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



✓ Early Stopping

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)), #hidden layer
    tf.keras.layers.Dense(32, activation='relu'), #hidden layer
    tf.keras.layers.Dense(1, activation='sigmoid') #output layer
])
```

```

model.compile(
    optimizer='adam',
    loss='binary_crossentropy', # loss for binary classification
    metrics=['accuracy'] # track accuracy
)

```

```

early_stop = tf.keras.callbacks.EarlyStopping(
    monitor='val_loss', # watch validation loss
    patience=3, # wait 3
    restore_best_weights=True
)

```

```

history = model.fit(
    x, y,
    epochs=100, # large number (training may stop early)
    batch_size=32,
    validation_split=0.2,
    callbacks=[early_stop], # apply early stopping
)

```

```

Epoch 1/100
25/25 ————— 1s 12ms/step - accuracy: 0.4988 - loss: 0.6900 - val_acc
Epoch 2/100
25/25 ————— 0s 4ms/step - accuracy: 0.4888 - loss: 0.6739 - val_accu
Epoch 3/100
25/25 ————— 0s 5ms/step - accuracy: 0.6250 - loss: 0.6589 - val_accu
Epoch 4/100
25/25 ————— 0s 5ms/step - accuracy: 0.6488 - loss: 0.6370 - val_accu
Epoch 5/100
25/25 ————— 0s 4ms/step - accuracy: 0.7437 - loss: 0.6095 - val_accu
Epoch 6/100
25/25 ————— 0s 5ms/step - accuracy: 0.7825 - loss: 0.5756 - val_accu
Epoch 7/100
25/25 ————— 0s 4ms/step - accuracy: 0.8213 - loss: 0.5267 - val_accu
Epoch 8/100
25/25 ————— 0s 4ms/step - accuracy: 0.8750 - loss: 0.4766 - val_accu
Epoch 9/100
25/25 ————— 0s 4ms/step - accuracy: 0.8888 - loss: 0.4261 - val_accu
Epoch 10/100
25/25 ————— 0s 4ms/step - accuracy: 0.8850 - loss: 0.3785 - val_accu
Epoch 11/100
25/25 ————— 0s 5ms/step - accuracy: 0.9137 - loss: 0.3345 - val_accu
Epoch 12/100
25/25 ————— 0s 4ms/step - accuracy: 0.9250 - loss: 0.3022 - val_accu
Epoch 13/100
25/25 ————— 0s 4ms/step - accuracy: 0.9388 - loss: 0.2717 - val_accu
Epoch 14/100
25/25 ————— 0s 4ms/step - accuracy: 0.9463 - loss: 0.2465 - val_accu
Epoch 15/100
25/25 ————— 0s 4ms/step - accuracy: 0.9500 - loss: 0.2280 - val_accu
Epoch 16/100
25/25 ————— 0s 4ms/step - accuracy: 0.9438 - loss: 0.2071 - val_accu
Epoch 17/100
25/25 ————— 0s 4ms/step - accuracy: 0.9575 - loss: 0.1935 - val_accu
Epoch 18/100
25/25 ————— 0s 4ms/step - accuracy: 0.9613 - loss: 0.1819 - val_accu
Epoch 19/100
25/25 ————— 0s 5ms/step - accuracy: 0.9650 - loss: 0.1662 - val_accu
Epoch 20/100
25/25 ————— 0s 4ms/step - accuracy: 0.9663 - loss: 0.1596 - val_accu
Epoch 21/100

```

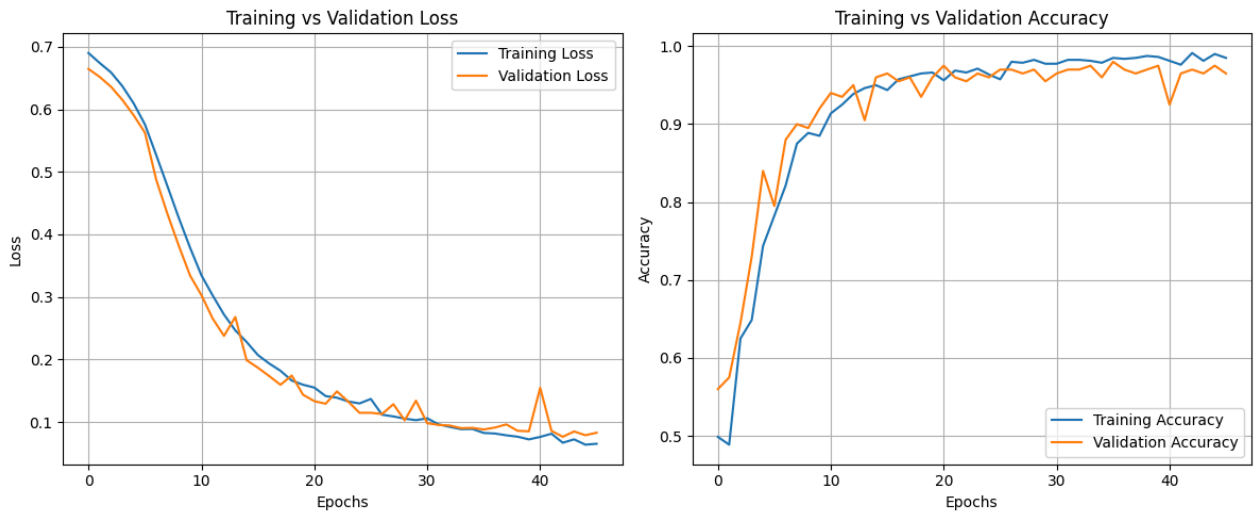
```
25/25 ————— 0s 5ms/step - accuracy: 0.9563 - loss: 0.1550 - val_accu
Epoch 22/100
25/25 ————— 0s 4ms/step - accuracy: 0.9688 - loss: 0.1414 - val_accu
Epoch 23/100
25/25 ————— 0s 4ms/step - accuracy: 0.9663 - loss: 0.1388 - val_accu
Epoch 24/100
25/25 ————— 0s 5ms/step - accuracy: 0.9712 - loss: 0.1328 - val_accu
Epoch 25/100
25/25 ————— 0s 4ms/step - accuracy: 0.9638 - loss: 0.1296 - val_accu
Epoch 26/100
25/25 ————— 0s 5ms/step - accuracy: 0.9575 - loss: 0.1368 - val_accu
Epoch 27/100
25/25 ————— 0s 4ms/step - accuracy: 0.9800 - loss: 0.1116 - val_accu
Epoch 28/100
25/25 ————— 0s 4ms/step - accuracy: 0.9787 - loss: 0.1088 - val_accu
Epoch 29/100
```

```
#Learning curves
train_loss = history.history['loss'] # training loss
val_loss = history.history['val_loss'] # validation loss
train_acc = history.history['accuracy'] # training accuracy
val_acc = history.history['val_accuracy'] # validation accuracy

plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```



✓ Dropout

```
model = tf.keras.Sequential([
    tf.keras.layers.Dense(64, activation='relu', input_shape=(10,)),
    tf.keras.layers.Dropout(0.5), # Drop 50% of neurons randomly
    tf.keras.layers.Dense(32, activation='relu'),
    tf.keras.layers.Dropout(0.3), # Drop 30% of neurons
    tf.keras.layers.Dense(1, activation='sigmoid')
])
```

```
model.compile(
    optimizer='adam',
    loss='binary_crossentropy', # loss for binary classification
    metrics=['accuracy'] # track accuracy
)

history = model.fit(
    x, y,
    epochs=20,
    batch_size=32,
    validation_split=0.2,
)

#Learning curves
train_loss = history.history['loss'] # training loss
val_loss = history.history['val_loss'] # validation loss
train_acc = history.history['accuracy'] # training accuracy
val_acc = history.history['val_accuracy'] # validation accuracy

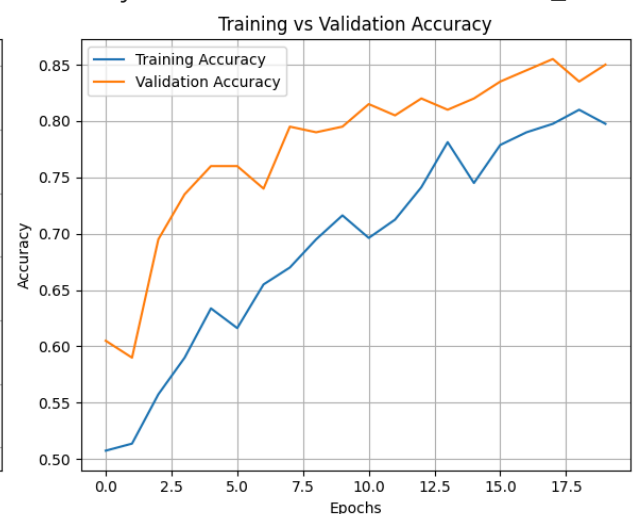
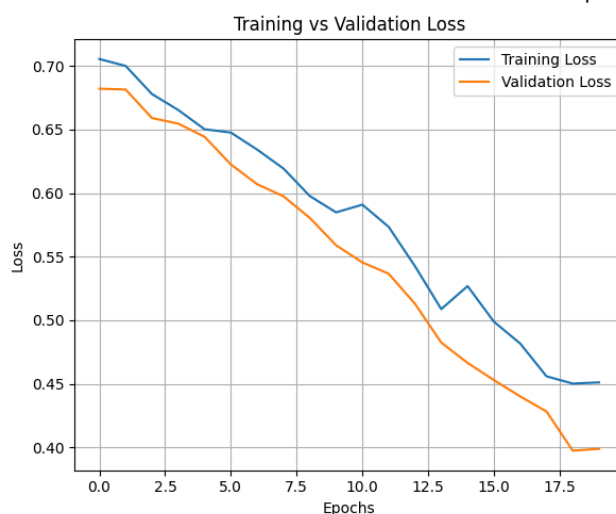
plt.figure(figsize=(12, 5))
plt.subplot(1, 2, 1)
plt.plot(train_loss, label='Training Loss')
plt.plot(val_loss, label='Validation Loss')
plt.title('Training vs Validation Loss')
plt.xlabel('Epochs')
plt.ylabel('Loss')
```

```
plt.legend()
plt.grid(True)

plt.subplot(1, 2, 2)
plt.plot(train_acc, label='Training Accuracy')
plt.plot(val_acc, label='Validation Accuracy')
plt.title('Training vs Validation Accuracy')
plt.xlabel('Epochs')
plt.ylabel('Accuracy')
plt.legend()
plt.grid(True)

plt.tight_layout()
plt.show()
```

Epoch 1/20
 25/25 ————— 2s 12ms/step - accuracy: 0.5075 - loss: 0.7054 - val_accu
 Epoch 2/20
 25/25 ————— 0s 5ms/step - accuracy: 0.5138 - loss: 0.7000 - val_accu
 Epoch 3/20
 25/25 ————— 0s 4ms/step - accuracy: 0.5575 - loss: 0.6779 - val_accu
 Epoch 4/20
 25/25 ————— 0s 5ms/step - accuracy: 0.5900 - loss: 0.6654 - val_accu
 Epoch 5/20
 25/25 ————— 0s 4ms/step - accuracy: 0.6338 - loss: 0.6502 - val_accu
 Epoch 6/20
 25/25 ————— 0s 4ms/step - accuracy: 0.6162 - loss: 0.6476 - val_accu
 Epoch 7/20
 25/25 ————— 0s 5ms/step - accuracy: 0.6550 - loss: 0.6342 - val_accu
 Epoch 8/20
 25/25 ————— 0s 4ms/step - accuracy: 0.6700 - loss: 0.6193 - val_accu
 Epoch 9/20
 25/25 ————— 0s 4ms/step - accuracy: 0.6950 - loss: 0.5977 - val_accu
 Epoch 10/20
 25/25 ————— 0s 4ms/step - accuracy: 0.7163 - loss: 0.5849 - val_accu
 Epoch 11/20
 25/25 ————— 0s 5ms/step - accuracy: 0.6963 - loss: 0.5909 - val_accu
 Epoch 12/20
 25/25 ————— 0s 4ms/step - accuracy: 0.7125 - loss: 0.5735 - val_accu
 Epoch 13/20
 25/25 ————— 0s 4ms/step - accuracy: 0.7412 - loss: 0.5425 - val_accu
 Epoch 14/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7812 - loss: 0.5088 - val_accu
 Epoch 15/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7450 - loss: 0.5269 - val_accu
 Epoch 16/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7788 - loss: 0.4989 - val_accu
 Epoch 17/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7900 - loss: 0.4817 - val_accu
 Epoch 18/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7975 - loss: 0.4560 - val_accu
 Epoch 19/20
 25/25 ————— 0s 6ms/step - accuracy: 0.8100 - loss: 0.4502 - val_accu
 Epoch 20/20
 25/25 ————— 0s 5ms/step - accuracy: 0.7975 - loss: 0.4512 - val_accu



✓ K-Fold Cross Validation

using scikit-learn


```

from sklearn.model_selection import KFold
from sklearn.linear_model import LogisticRegression
from sklearn.datasets import load_iris
from sklearn.metrics import accuracy_score

```

```

data = load_iris()
X, y = data.data, data.target

```

```

kf = KFold(n_splits=5)
model = LogisticRegression(max_iter=200)
scores = []

for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]
    model.fit(X_train, y_train)
    predictions = model.predict(X_test)
    scores.append(accuracy_score(y_test, predictions))

print("Cross-validation scores:", scores)
print("Average score:", sum(scores)/len(scores))

```

```

Cross-validation scores: [1.0, 1.0, 0.8666666666666667, 0.9333333333333333, 0.8333333333333333]
Average score: 0.9266666666666667

```

using tensorflow

```

kf = KFold(n_splits=5, shuffle=True, random_state=42)
scores = []
for train_index, test_index in kf.split(X):
    X_train, X_test = X[train_index], X[test_index]
    y_train, y_test = y[train_index], y[test_index]

    model = tf.keras.Sequential([
        tf.keras.layers.Dense(16, activation='relu', input_shape=(X.shape[1],)),
        tf.keras.layers.Dense(8, activation='relu'),
        tf.keras.layers.Dense(3, activation='softmax') # 3 classes in Iris
    ])

    model.compile(
        optimizer='adam',
        loss='sparse_categorical_crossentropy',
        metrics=['accuracy']
    )

    model.fit(X_train, y_train, epochs=30, batch_size=16)
    loss, accuracy = model.evaluate(X_test, y_test)
    scores.append(accuracy)

print("Cross-validation scores:", scores)
print("Average score:", np.mean(scores))

```

```

Epoch 1/30
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning:
    super().__init__(activity_regularizer=activity_regularizer, **kwargs)
8/8 1s 7ms/step - accuracy: 0.3333 - loss: 1.5512
Epoch 2/30

```

8/8  0s 5ms/step - accuracy: 0.3333 - loss: 1.3741
Epoch 3/30
8/8  0s 5ms/step - accuracy: 0.4167 - loss: 1.2317
Epoch 4/30
8/8  0s 5ms/step - accuracy: 0.6500 - loss: 1.1254
Epoch 5/30
8/8  0s 6ms/step - accuracy: 0.6583 - loss: 1.0417
Epoch 6/30
8/8  0s 5ms/step - accuracy: 0.6583 - loss: 0.9735
Epoch 7/30
8/8  0s 5ms/step - accuracy: 0.6583 - loss: 0.9202
Epoch 8/30